```
typedef struct LNode
{
    int data
    struct LNode *next;
}{LNode; LinkList;

LNode * Link list
```

```
LinkList Connect(LinkList ha, LinkList hb, int m, int n)
{
    LinkList hc=NULL;
    LinkList pTail=NULL;
    pTail=ha;
    while(pTail!=NULL
    {
        pTail=pTail->next;
    }
    pTail->next = hb->next
    free(hb)
    hc=ha;
    return hc;
}
```

时间复杂度O(m)

无论如何只遍历a链表

~~void Locate(L, x)~~

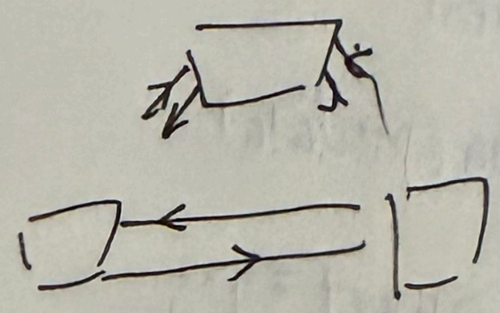
```
void Locate(List L, int x)
{
    ~~Node~~ List select(List a, int mink, int maxk)
    {
        if(a==NULL || a->next==NULL || mink>=maxk)
            return;
        List pre = a;
        List present = a->next;
        List q = NULL;
        while(present != NULL && present->data <= mink)
        {
            pre = present
            present = present->next;
        }
        while(present != NULL && present->data < maxk)
        {
            q = present
            p = p->next
            present = present->next
            free(q)
        }
        pre->next = present;
    }
```

时间复杂度 O(mink+maxk)

~~void Locate(L, x)~~

```
void Locate(List L, int x)
{
   Node *p = L->next;
   while (p!=L && p->data != x)
   {  p= p->next;
  {
  if(p==L)
     return NULL;
  p->freq++;

  p-> prior->next = p->next
  p->next->prior = p->prior
  Node *q = p->prior;
  while (q!=L && q->freq <= p->freq)
  {  q = p->prior;
  {
  p->next = q->next
  q->next->prior = p;
  p->qriur = q;
  q->next = p
```



```
{
```

```
void Split (List L, List *La, List *Lb, List *Lc)
{
    *La = (List) malloc (sizeof(LNode));
     (*La)->next = *La;
    *Lb = (List) malloc (sizeof(LNode));
     (*Lb)->next = *Lb;
    *Lc = (List) malloc (sizeof(LNode));
     (*Lc)->next = *Lc

    List tailA = *La;
    List tailB = *Lb;
    List tailC = *Lc;

    List p = L->next;
     List q = NULL;
    while (p!=NULL)
    { q= p->next
      if(isalpha(p->data)
      {  tailA->next = p;
         tailA = p;
    { elseif ( isdigit (p->data)
      { tailB->next = p;
        tailB = p;
      { else
      { tailC->next = p;
        tailC = p;
      {
    {
```

```
tailA->next = *La
tail B->next = *Lb
tail C->next = *Lc
L->next = NULL
```

```
void combine(Data &a, Data &b, Data &c)
{
    int idx_c = 0;
    int idx_a[a.length] = {0}, idx_b[a.length] = {0};
    for(int i = 0; i < a.length; i++)
    {
        for(int j = i; j < a.length; j++)
        {
            if(a[i] == a[j])
            {
                idx_a[j] = 1;
            }
        }
    }
    for( i = 0; i < b.length; i++)
    {
        for(j = i; j < b.length; j++)
        {
            if(b[i] == b[j])
            {
                idx_b[j] = 1;
            }
        }
    }

    for(i = 0; i < a.length; i++)
    {
        if(idx_a[i] == 1)
        {
            continue;
        }
        for(j = 0; j < b.length; j++)
        {
            if(idx[b] == 1)
            {
                continue;
            }
            if(a.ele[i] == b.ele[j])
            {
                c.ele[idx_c] = a.ele[i];
                idx_c++;
            }
        }
    }
}
```

2.11

```
void add_ele (Data &a, int pos, Data b)
{
  v.push_back(1);
~~for(in~~ a.length++;
  for(int i = a.length ; a>=pos; a--)
  {
    a.ele[i] = a.ele[i-1];
  {   a.ele[i-1] = 0;

  a.ele[pos] = b;
{
```

2.21 void reverse (Data &a)

```
{
  int half_len = (int)a.length / 2
  for(int i = 0; i< half_len; i++)
  {
    ~~int temp_end_idx = half_len a.length-i~~
    int temp_ele = ~~a.length~~ a.ele[a.length-i];
    a.ele[a.length-i] = a.ele[i];
    a.ele[i] = temp_ele;
  }
}
```

2.10

Status DeleK(SqlistR a, int i, int k)

if (i<1 || k<0 || i+k>a.length)

↳ return INFEASIBLE;

else

{ count=k-i; count=i+1

for(int ~~count=i+1, con~~ count<=k; count++)

{

~~a.elem[count~~ for(int j=count, j<a.length, j++)

{ a.elem[j] = a.elem[j+1];

a.length--

}

}

return 0;